Министерство образования и науки Российской Федерации
Санкт-Петербургский Политехнический Университет Петра Великого
—
Высшая школа кибербезопасности

ЛАБОРАТОРНАЯ РАБОТА № 4

«Разработка, интеграция, тестирование и развертывание сложного программного обеспечения. Часть 2»

по дисциплине «Технологии разработки современного программного обеспечения»

Выполнили
студент гр.5151003/30801 Трифанов А.С.
<подпись>

студент гр.5151003/30801 Игнатов Д.П.
<подпись>

Преподаватель
Ассистент преподавателя Проурзин О.В.
<подпись>

Санкт-Петербург
2026

1. Цель работы

Получение практических навыков разработки компонентов с дополнительной бизнес-логикой и их интеграции в сложное программного обеспечения, а также построения конвейера в рамках CI/CD.

2. Задачи

В рамках выполнения лабораторной работы необходимо основываясь на результатах лабораторной работы №3 решить задачи, указанные в приложении для выбранной темы.

3. Ход работы

3.1. Контейнеризация и публикация образов

Первым этапом практической реализации стала подготовка изолированных сред выполнения для компонентов проекта LabServer. Для обеспечения независимости от окружения и упрощения последующего развертывания исходный код приложений был упакован в Docker-контейнеры.

Сборка образов осуществлялась с применением паттерна Multi-stage builds (многоэтапная сборка), что является современной практикой написания Dockerfile. Данный подход позволяет разделить процессы компиляции кода и его выполнения:

— Для серверной части (Backend): На базовом этапе сборки использовался полновесный образ .NET 9.0 SDK для восстановления зависимостей и компиляции исходного кода ASP.NET Core. На финальном этапе скомпилированные бинарные файлы копировались в минималистичный образ .NET ASP.NET 9.0 Runtime.

— Для клиентских приложений (Dashboard и Admin): В процессе сборки Angular-приложений использовался образ на базе Node.js. На этапе подготовки производилась установка зависимостей пакетным менеджером npm и сборка проектов для последующего обслуживания запросов сервером.

Использование многоэтапной сборки позволило существенно оптимизировать итоговый размер Docker-образов за счет исключения из них исходного кода, тяжеловесных

сборочных утилит и лишних зависимостей. Это не только ускоряет процесс передачи образов по сети, но и снижает поверхность потенциальных уязвимостей в рабочей среде.

После успешной локальной сборки компонентам были назначены версионные теги, и они были отправлены (push) в публичный облачный реестр контейнеров Docker Hub под учетной записью trialextime. Были опубликованы следующие образы:

— trialextime/labserver-backend:v1

— trialextime/labserver-dashboard:v1

— trialextime/labserver-admin:v1

Публикация в централизованном реестре является обязательным шагом для построения CI/CD конвейера. Это гарантирует, что агенты Kubernetes (kubelet) смогут беспрепятственно скачивать актуальные версии компонентов на любые узлы кластера при развертывании инфраструктуры.

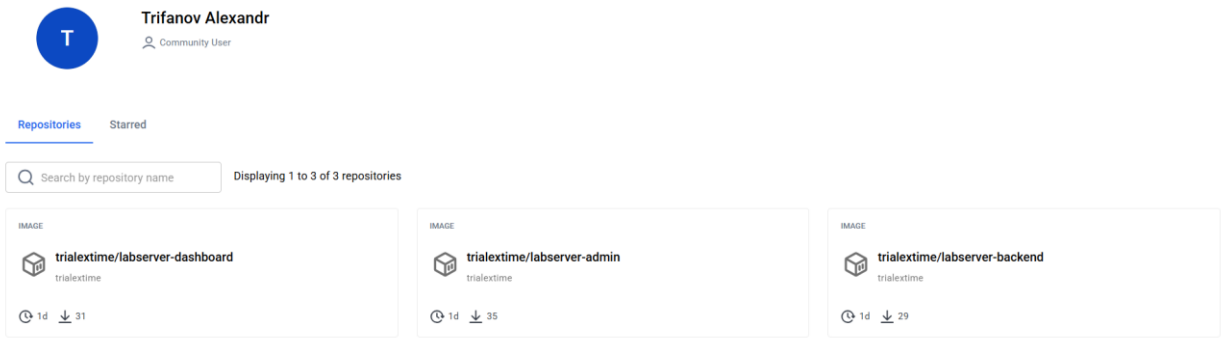

Рисунок 3.1 — Опубликованные образы компонентов LabServer в публичном реестре Docker Hub

3.2. Развертывание инфраструктуры в Kubernetes

После успешной контейнеризации компонентов была развернута целевая инфраструктура в среде оркестрации. Для симуляции производственного окружения на локальной машине был запущен кластер Kubernetes с использованием инструмента Minikube на базе драйвера Docker.

Процесс развертывания был полностью автоматизирован с применением декларативных манифестов, разработанных на этапе проектирования (ЛР №3):

— db.yaml — манифест для создания экземпляра СУБД PostgreSQL.

— app.yaml — манифест для запуска серверной и клиентской частей LabServer.

Манифесты были применены в кластер с помощью штатной утилиты kubectl apply. Kubernetes, в свою очередь, самостоятельно скачал ранее опубликованные Docker-образы из реестра Docker Hub и запустил их в изолированных подах (Pods).

Для обеспечения сетевой связности между компонентами были использованы следующие объекты Service:

— Сервис db-service типа ClusterIP был создан для PostgreSQL. Это позволило присвоить базе данных стабильное внутреннее DNS-имя. Бэкенд-приложение теперь может подключаться к СУБД по этому имени, не завися от динамически изменяемого IP-адреса пода базы данных.

— Сервис dashboard-service типа NodePort был сконфигурирован для фронтенд-приложения. Он открывает порт 30000 на всех узлах кластера и пробрасывает трафик на порт 4200 контейнера, делая веб-интерфейс доступным для конечного пользователя через браузер.

После применения манифестов и автоматического скачивания образов система успешно перешла в стабильное состояние, что подтверждается статусом Running для всех запущенных подов.

```
PS C:\Temp\devops-lab-17> kubectl get pods
NAME                          READY   STATUS    RESTARTS      AGE
backend-7dbd976b69-f546k      1/1     Running   1 (29m ago)   4h
dashboard-5f478c95f7-6kckq    1/1     Running   3 (24m ago)   27h
postgres-849fdff8d4-4db6k     1/1     Running   2 (29m ago)   27h
```

Рисунок 3.2 — Статус запущенных подов LabServer в кластере Kubernetes

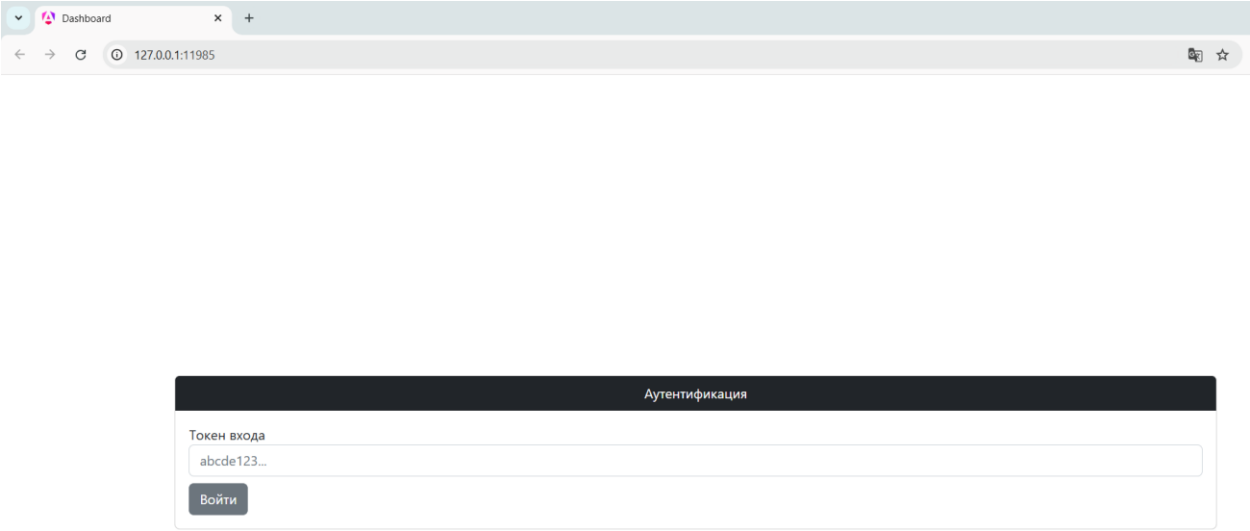

Рисунок 3.3 — Доступ к клиентскому приложению через сервис типа NodePort

Доступ к приложению был получен с помощью команды minikube service, которая автоматически создает туннель к сервису, как показано на Рисунке 3.3

3.3. Практическая реализация Liveness Probes (Тема №17)

Для обеспечения высокой доступности и автоматического восстановления системы после сбоев были реализованы механизмы мониторинга живучести на базе встроенного инструмента Kubernetes — Liveness Probes. Данные проверки интегрированы непосредственно в манифест развертывания приложения (app.yaml) и выполняются агентом kubelet на каждом узле кластера.

В ходе практической реализации были внедрены проверки состояния контейнеров на уровне транспортного протокола TCP. Для этого использовался тип пробы tcpSocket, который проверяет способность приложения принимать TCP-соединения на указанном порту. Этот метод позволяет надежно отслеживать работоспособность сервиса без необходимости модификации его исходного кода для добавления специальных HTTP-эндпоинтов (/health).

Ключевым аспектом настройки Liveness Probes стало обоснование и подбор временных параметров (initialDelaySeconds), чтобы избежать ложных срабатываний и попадания подов в цикл бесконечных перезапусков (CrashLoopBackOff):

— Для серверной части:

Установлена задержка перед первой проверкой в 60 секунд. Этот интервал необходим для того, чтобы контейнер с СУБД PostgreSQL успел полностью инициализироваться. Только после этого бэкенд может успешно установить соединение с базой данных и запустить веб-сервер Kestrel, который, в свою очередь, откроет порт 5288 для входящих соединений.

— Для клиентской части:

Установлена задержка в 120 секунд. Так как фронтенд-приложение на Angular запускается в режиме разработки, при старте контейнера происходит длительный процесс компиляции TypeScript-кода и сборки модулей с помощью Webpack. Задержка в 2 минуты гарантирует, что kubelet не начнет опрашивать порт 4200 до того, как веб-сервер Angular будет полностью готов принимать трафик.

Данные настройки обеспечили стабильный запуск системы и корректную работу механизма самовосстановления. Конфигурация Liveness Probes была верифицирована с помощью команды kubectl describe pod.

```
PS C:\Temp\devops-lab-17> kubectl describe pod backend-7dbd976b69-f546k
Name:             backend-7dbd976b69-f546k
Namespace:        default
Priority:         0
Service Account:  default
Node:             minikube/192.168.49.2
Start Time:       Sun, 26 Apr 2026 12:58:22 +0300
Labels:           app=backend
                  pod-template-hash=7dbd976b69
Annotations:      <none>
Status:           Running
IP:               10.244.0.40
IPs:
  IP:           10.244.0.40
Controlled By:  ReplicaSet/backend-7dbd976b69
Containers:
  backend:
    Container ID:   docker://28b0750044c8723b623158b069c201e16fc7719a8c83b0351b6560cf03023e551
    Image:          trialextime/labserver-backend:v1
    Image ID:       docker-pullable://trialextime/labserver-backend@sha256:b96f14df2a4a232d8132ecb1dcc651b2c37a53decdafa9e174ef9d242f09a882
    Port:           5288/TCP
    Host Port:      0/TCP
    State:          Running
      Started:      Sun, 26 Apr 2026 16:30:44 +0300
    Last State:     Terminated
      Reason:       Error
      Exit Code:    255
      Started:      Sun, 26 Apr 2026 12:58:26 +0300
      Finished:     Sun, 26 Apr 2026 16:28:37 +0300
    Ready:          True
    Restart Count:  1
    Liveness:       tcp-socket :5288 delay=60s timeout=1s period=15s #success=1 #failure=3
    Environment:
      ConnectionStrings__PgSQL:  Host=db-service;Port=5432;Database=labs;Username=postgres;Password=changeit
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-2sf9r (ro)
Conditions:
  Type                        Status
  PodReadyToStartContainers   True
  Initialized                 True
  Ready                       True
  ContainersReady             True
  PodScheduled                True
Volumes:
  kube-api-access-2sf9r:
    Type:                    Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds:  3607
    ConfigMapName:           kube-root-ca.crt
    Optional:                false
    DownwardAPI:             true
QoS Class:                   BestEffort
Node-Selectors:              <none>
Tolerations:                 node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                             node.kubernetes.io/unreachable:NoExecute op=Exists for 300s

Events:
  Type    Reason          Age    From               Message
  ----    ------          ----   ----               -------
  Normal  Scheduled       3h46m  default-scheduler  Successfully assigned default/backend-7dbd976b69-f546k to minikube
  Normal  Pulled          3h46m  kubelet            Container image "trialextime/labserver-backend:v1" already present on machine and can be accessed by the pod
  Normal  Created         3h46m  kubelet            Container created
  Normal  Started         3h46m  kubelet            Container started
  Normal  SandboxChanged  14m    kubelet            Pod sandbox changed, it will be killed and re-created.
  Normal  Pulled          14m    kubelet            Container image "trialextime/labserver-backend:v1" already present on machine and can be accessed by the pod
  Normal  Created         14m    kubelet            Container created
  Normal  Started         14m    kubelet            Container started
```

Рисунок 3.4 — Подтверждение конфигурации правил Liveness Probe для пода серверной части

3.4. Развертывание стека мониторинга

Для практической реализации подсистемы мониторинга был развернут стек Prometheus + Grafana. Вместо ручного написания большого набора Kubernetes-манифестов был использован пакетный менеджер Helm, что соответствует промышленной практике развертывания составных сервисов в Kubernetes.

Мониторинг был установлен в отдельное пространство имен monitoring. Это позволяет логически отделить служебные компоненты наблюдаемости от прикладных подов LabServer, которые работают в пространстве имен default.

Команды развертывания стека мониторинга:

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts

helm repo update

kubectl create namespace monitoring

helm install monitoring prometheus-community/kube-prometheus-stack --namespace monitoring --set grafana.adminPassword=admin12345
```

В результате установки Helm-чарт kube-prometheus-stack развернул набор взаимосвязанных компонентов: Prometheus Operator, Prometheus, Grafana, kube-state-metrics, node-exporter и Alertmanager. Данный набор закрывает основные задачи мониторинга Kubernetes-кластера: сбор метрик, хранение временных рядов, визуализацию и подготовку к настройке уведомлений.

Особое значение для данной лабораторной работы имеет компонент kube-state-metrics. Он экспортирует состояние объектов Kubernetes в виде метрик Prometheus. Благодаря этому становится возможным отслеживать фазы подов, количество рестартов контейнеров, состояние deployment-ресурсов и другие параметры без изменения исходного кода приложения.

Корректность установки проверялась командой:

```
kubectl get pods -n monitoring
```

В пространстве имен monitoring были запущены следующие служебные поды: monitoring-grafana, monitoring-kube-prometheus-operator, monitoring-kube-state-metrics, monitoring-prometheus-node-exporter, prometheus-monitoring-kube-prometheus-prometheus-0 и alertmanager-monitoring-kube-prometheus-alertmanager-0. После завершения загрузки контейнеров все они перешли в состояние Running.

```
PS C:\dev\devops-lab-17> kubectl get pods -n monitoring
NAME                                                     READY   STATUS    RESTARTS      AGE
alertmanager-monitoring-kube-prometheus-alertmanager-0   2/2     Running   2 (17h ago)   19h
monitoring-grafana-75c4bc8cd8-79ffq                      3/3     Running   3 (17h ago)   19h
monitoring-kube-prometheus-operator-7c8586f484-mz7xd     1/1     Running   1 (17h ago)   19h
monitoring-kube-state-metrics-67d5f7bf68-m9z22           1/1     Running   1 (17h ago)   19h
monitoring-prometheus-node-exporter-s2njf                1/1     Running   1 (17h ago)   19h
prometheus-monitoring-kube-prometheus-prometheus-0       2/2     Running   2 (17h ago)   19h
```

Рисунок 3.5 — Список запущенных подов стека мониторинга в пространстве имен monitoring

3.5. Визуализация метрик и настройка Dashboard

После установки kube-prometheus-stack интерфейс Grafana был открыт через локальный проброс порта сервиса monitoring-grafana:

```
kubectl port-forward -n monitoring svc/monitoring-grafana 3000:80
```

После выполнения данной команды веб-интерфейс Grafana стал доступен по адресу http://localhost:3000. В качестве источника данных использовался Prometheus, автоматически созданный и подключенный в Grafana при установке Helm-чарта.

Для проверки доступа к метрикам был использован режим Explore. В нем был выполнен запрос к метрике kube_pod_status_phase, которая экспортируется компонентом kube-state-metrics и показывает фазу жизненного цикла подов Kubernetes.

Основные PromQL-запросы, использованные на информационной панели:

1. Количество работающих подов проекта LabServer:

```
sum(kube_pod_status_phase{phase="Running", namespace="default", pod=~"backend.*|dashboard.*|postgres.*"})
```

Данный запрос суммирует все поды LabServer, находящиеся в состоянии Running. При штатной работе значение равно 3, так как запущены backend, dashboard и postgres.

2. Статус каждого пода отдельно:

```
kube_pod_status_phase{phase="Running", namespace="default", pod=~"backend.*|dashboard.*|postgres.*"}
```

Результат запроса используется для панели Pod Running Status. Значение 1 означает, что конкретный под находится в состоянии Running, а значение 0 означает, что он не находится в этой фазе.

3. Счетчик рестартов контейнеров:

```
sum by (pod) (kube_pod_container_status_restarts_total{namespace="default", pod=~"backend.*|dashboard.*|postgres.*"})
```

Данный запрос показывает количество перезапусков контейнеров по каждому поду. Он позволяет зафиксировать последствия сбоев и подтвердить работу механизма самовосстановления Kubernetes.

4. Количество подов в проблемных состояниях:

```
sum(kube_pod_status_phase{phase=~"Pending|Failed|Unknown", namespace="default", pod=~"backend.*|dashboard.*|postgres.*"})
```

Данный запрос используется для панели Problem pods. При нормальной работе системы его значение равно 0.

На основе указанных запросов была создана информационная панель LabServer Kubernetes Monitoring. Она содержит четыре ключевые панели: Running pods LabServer, Pod Running Status, Container restarts и Problem pods. Такой набор позволяет быстро оценить текущее состояние системы, увидеть количество работающих подов, проверить каждый компонент отдельно и обнаружить рестарты контейнеров.

После настройки dashboard был экспортирован в JSON-файл grafana/labserver-k8s-monitoring-dashboard.json и добавлен в репозиторий проекта. Это позволяет импортировать ту же панель мониторинга на другом компьютере и воспроизвести конфигурацию без ручной настройки. Финальный дашборд Grafana продемонстрирован на Рисунке 3.6:

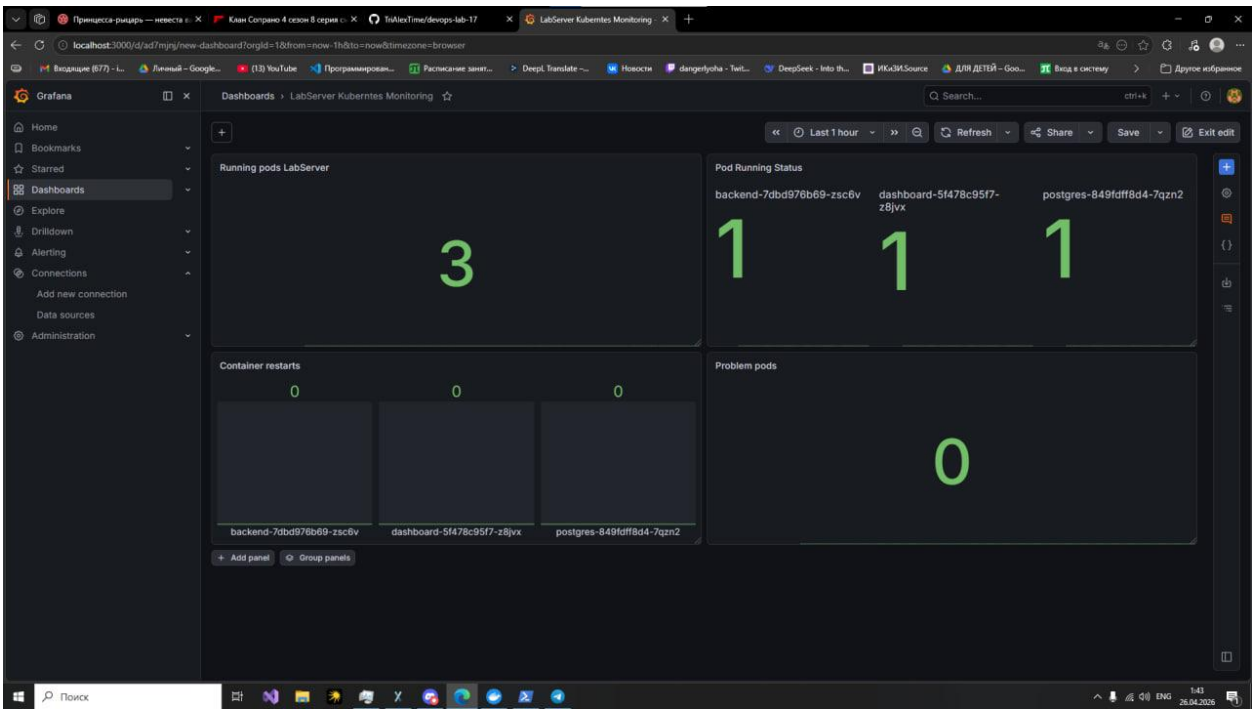

Рисунок 3.6 — Финальный дашборд Grafana для мониторинга состояния компонентов LabServer

3.6. Тестирование отказоустойчивости системы

Финальным этапом лабораторной работы стало проведение комплексного тестирования, целью которого была проверка работоспособности реализованных механизмов самовосстановления и реакции системы мониторинга на инциденты. Для этого был проведен «краш-тест», симулирующий нештатное завершение работы одного из ключевых компонентов системы.

Сценарий тестирования:

1. **Начальное состояние:**

Система находится в стабильном состоянии. Дашборд в Grafana показывает, что все 3 пода проекта LabServer (backend, dashboard, postgres) находятся в статусе Running.

2. **Инициация инцидента:**

С помощью утилиты kubectl был принудительно удален под, отвечающий за работу серверной части приложения, по команде:

kubectl delete pod -l app=backend.

3. **Реакция системы оркестрации:**

Контроллер Kubernetes (Deployment) мгновенно зафиксировал несоответствие текущего состояния кластера желаемому (отсутствие одной реплики бэкенда). В ответ на это был немедленно запущен процесс создания нового пода для замены, вышедшего из строя.

4. **Реакция системы мониторинга:**

Система Prometheus, собирающая метрики состояния (kube_pod_status_phase), зафиксировала исчезновение одного пода из пула работающих. Это изменение было передано в Grafana, что привело к немедленной визуальной реакции на информационной панели:

- Общее количество работающих подов временно уменьшилось.
- График состояния показал провал и последующее восстановление.
- После запуска нового пода счетчик рестартов для соответствующего компонента увеличился, сигнализируя администратору о прошедшем инциденте.

Тестирование продемонстрировало, что спроектированная инфраструктура обладает высокой степенью отказоустойчивости. Механизмы Kubernetes обеспечили автоматическое восстановление работоспособности сервиса без ручного вмешательства, а связка Prometheus и Grafana позволила прозрачно отследить весь жизненный цикл инцидента.

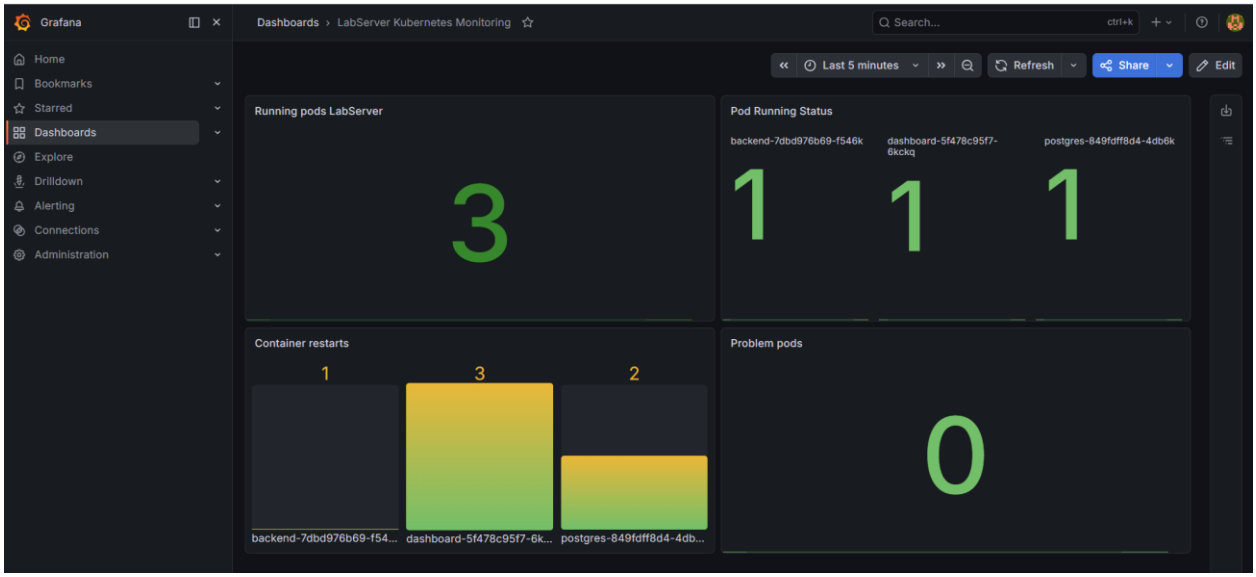

Рисунок 3.7 — Состояние системы мониторинга в штатном режиме работы

Рисунок 3.8 — Реакция системы мониторинга: Kubernetes автоматически создал новый экземпляр Backend после удаления старого

4. Выводы

В ходе выполнения лабораторной работы была успешно решена задача развертывания сложного многокомпонентного программного обеспечения в современной cloud-native среде.

На основе проведенного анализа были разработаны и применены декларативные манифесты для оркестратора Kubernetes, позволившие автоматизировать процесс запуска и

конфигурирования системы LabServer, включая серверную часть, клиентское приложение Dashboard и базу данных PostgreSQL. Для обеспечения сетевой связности были использованы сервисы ClusterIP и NodePort, что позволило организовать как внутреннее взаимодействие компонентов, так и внешний доступ к веб-интерфейсу.

Ключевым результатом инфраструктурной части стала практическая реализация задачи темы №17 — настройка контроля живучести контейнеров. За счет внедрения Liveness Probes с индивидуально подобранными задержками и TCP-проверками портов 5288 и 4200 была создана самовосстанавливающаяся система, способная автоматически обнаруживать и перезапускать отказавшие сервисы.

В рамках части работы, связанной с мониторингом, был развернут стек Prometheus + Grafana с использованием Helm-чарта kube-prometheus-stack в отдельном пространстве имен monitoring. Такой способ установки позволил быстро получить готовую инфраструктуру наблюдаемости, включающую Prometheus, Grafana, kube-state-metrics, node-exporter и Alertmanager.

Компонент kube-state-metrics обеспечил сбор метрик состояния объектов Kubernetes, включая kube_pod_status_phase и kube_pod_container_status_restarts_total. На основе этих метрик в Grafana был создан dashboard LabServer Kubernetes Monitoring, отображающий количество работающих подов, статус каждого компонента, количество рестартов контейнеров и наличие проблемных состояний.

Итоговая система мониторинга показала, что все три пода проекта LabServer находятся в состоянии Running, проблемных подов нет, а состояние системы может быть наглядно отслежено через Grafana. Экспортированный JSON-файл dashboard был добавлен в репозиторий, что обеспечивает воспроизводимость настройки мониторинга на другой машине участника команды.

5. Приложения

ПРИЛОЖЕНИЕ А: Ссылка на публичный GitHub-репозиторий

Репозиторий содержит исходный код проекта, конфигурационные манифесты Kubernetes и экспортированный дашборд Grafana.

https://github.com/TriAlexTime/devops-lab-17

ПРИЛОЖЕНИЕ Б: Листинги конфигурационных манифестов Kubernetes

Листинг Б.1 — Исходный код манифеста базы данных (db.yaml)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: postgres
spec:
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
      - name: postgres
        image: postgres:15
        env:
        - name: POSTGRES_PASSWORD
          value: "changeit"
        - name: POSTGRES_DB
          value: "labs"
        ports:
        - containerPort: 5432
---
apiVersion: v1
kind: Service
metadata:
  name: db-service
```

```
spec:
  selector:
    app: postgres
  ports:
  - port: 5432
```

Листинг Б.2 — Исходный код манифеста приложения (app.yaml)

```
# БЭКЕНД
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
spec:
  replicas: 1
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
      - name: backend
        image: trialextime/labserver-backend:v1
        ports:
        - containerPort: 5288
        env:
        - name: ConnectionStrings__PgSQL
          value: "Host=db-service;Port=5432;Database=labs;Username=postgres;Password=changeit"
        livenessProbe:
          tcpSocket:
            port: 5288
          initialDelaySeconds: 60
          periodSeconds: 15
---
apiVersion: v1
kind: Service
metadata:
```

```
  name: backend-service
spec:
  selector:
    app: backend
  ports:
  - port: 80
    targetPort: 5288

---
# ФРОНТЕНД - ДАШБОРД
apiVersion: apps/v1
kind: Deployment
metadata:
  name: dashboard
spec:
  selector:
    matchLabels:
      app: dashboard
  template:
    metadata:
      labels:
        app: dashboard
    spec:
      containers:
      - name: dashboard
        image: trialextime/labserver-dashboard:v1
        ports:
        - containerPort: 4200
        livenessProbe:
          tcpSocket:
            port: 4200
          initialDelaySeconds: 120
          periodSeconds: 15
---
apiVersion: v1
kind: Service
metadata:
  name: dashboard-service
spec:
  selector:
```

```
  app: dashboard
 type: NodePort
 ports:
 - port: 80
   targetPort: 4200
   nodePort: 30000
```

ПРИЛОЖЕНИЕ В: **Экспортированный JSON-файл конфигурации дашборда Grafana**


```
{
  "annotations": {
        "list": [
            {
              "builtIn": 1,
              "datasource": {
                  "type": "grafana",
                  "uid": "-- Grafana --"
              },
              "enable": true,
              "hide": true,
              "iconColor": "rgba(0, 211, 255, 1)",
              "name": "Annotations \u0026 Alerts",
              "type": "dashboard"
            }
          ]
      },
  "editable": true,
  "fiscalYearStartMonth": 0,
  "graphTooltip": 0,
  "id": 3527366762704896,
  "liveNow": false,
  "panels": [
      {
        "datasource": {
            "type": "prometheus",
            "uid": "prometheus"
          },
        "fieldConfig": {
            "defaults": {
                "color": {
```

```
                    "mode": "thresholds"
                },
                "thresholds": {
                    "mode": "absolute",
                    "steps": [
                        {
                            "color": "green",
                            "value": 0
                        },
                        {
                            "color": "red",
                            "value": 80
                        }
                    ]
                }
            }
        },
    "gridPos": {
        "h": 8,
        "w": 12,
        "x": 0,
        "y": 0
    },
    "id": 1,
    "options": {
        "colorMode": "value",
        "graphMode": "area",
        "justifyMode": "auto",
        "orientation": "auto",
        "percentChangeColorMode": "standard",
        "reduceOptions": {
            "calcs": [
                "lastNotNull"
            ],
            "fields": "",
            "values": false
        },
        "showPercentChange": false,
        "textMode": "auto",
        "wideLayout": true
```

```
                },
            "pluginVersion": "13.0.1",
            "targets": [
                {
                    "datasource": {
                        "type": "prometheus",
                        "uid": "prometheus"
                    },
                    "editorMode": "code",
                    "expr": "sum(kube_pod_status_phase{phase=\"Running\", namespace=\"default\", pod=~\"backend.*|dashboard.*|postgres.*\"})",
                    "legendFormat": "__auto",
                    "range": true,
                    "refId": "A"
                }
            ],
            "title": "Running pods LabServer",
            "type": "stat"
        },
        {
            "datasource": {
                "type": "prometheus",
                "uid": "prometheus"
            },
            "fieldConfig": {
                "defaults": {
                    "color": {
                        "mode": "thresholds"
                    },
                    "thresholds": {
                        "mode": "absolute",
                        "steps": [
                            {
                                "color": "green",
                                "value": 0
                            },
                            {
                                "color": "red",
                                "value": 80
                            }
```

```
                                        ]
                                    }
                                }
                            },
                "gridPos": {
                    "h": 8,
                    "w": 12,
                    "x": 12,
                    "y": 0
                },
                "id": 2,
                "options": {
                    "colorMode": "value",
                    "graphMode": "area",
                    "justifyMode": "auto",
                    "orientation": "auto",
                    "percentChangeColorMode": "standard",
                    "reduceOptions": {
                        "calcs": [
                            "lastNotNull"
                        ],
                        "fields": "",
                        "values": false
                    },
                    "showPercentChange": false,
                    "textMode": "auto",
                    "wideLayout": true
                },
                "pluginVersion": "13.0.1",
                "targets": [
                    {
                        "datasource": {
                            "type": "prometheus",
                            "uid": "prometheus"
                        },
                        "editorMode": "code",
                        "expr": "kube_pod_status_phase{phase=\"Running\", namespace=\"default\", pod=~\"backend.*|dashboard.*|postgres.*\"}",
                        "interval": "",
                        "legendFormat": "{{pod}}",
```

```
                "range": true,
                "refId": "A"
            }
        ],
    "title": "Pod Running Status",
    "type": "stat"
},
{
    "datasource": {
                "type": "prometheus",
                "uid": "prometheus"
            },
    "fieldConfig": {
                "defaults": {
                        "color": {
                            "mode": "thresholds"
                        },
                        "thresholds": {
                            "mode": "absolute",
                            "steps": [
                                {
                                    "color": "green",
                                    "value": 0
                                },
                                {
                                    "color": "red",
                                    "value": 80
                                }
                            ]
                        }
                    }
            },
    "gridPos": {
            "h": 8,
            "w": 12,
            "x": 0,
            "y": 8
        },
    "id": 3,
    "options": {
```

"displayMode": "gradient",
"legend": {
"calcs": [
],
"displayMode": "list",
"placement": "bottom",
"showLegend": false
},
"maxVizHeight": 300,
"minVizHeight": 16,
"minVizWidth": 8,
"namePlacement": "auto",
"orientation": "auto",
"reduceOptions": {
"calcs": [
"lastNotNull"
],
"fields": "",
"values": false
},
"showUnfilled": true,
"sizing": "auto",
"valueMode": "color"
},
"pluginVersion": "13.0.1",
"targets": [
{
"datasource": {
"type": "prometheus",
"uid": "prometheus"
},
"editorMode": "code",
"expr": "sum by (pod) (kube_pod_container_status_restarts_total{namespace=\"default\", pod=~\"backend.*|dashboard.*|postgres.*\"})",
"legendFormat": "__auto",
"range": true,
"refId": "A"
}
],

```
    "title": "Container restarts",
    "type": "bargauge"
  },
  {
    "datasource": {
        "type": "prometheus",
        "uid": "prometheus"
      },
    "fieldConfig": {
        "defaults": {
            "color": {
                "mode": "thresholds"
              },
            "thresholds": {
                "mode": "absolute",
                "steps": [
                  {
                    "color": "green",
                    "value": 0
                  },
                  {
                    "color": "red",
                    "value": 80
                  }
                ]
              }
            }
      },
    "gridPos": {
      "h": 8,
      "w": 12,
      "x": 12,
      "y": 8
    },
    "id": 4,
    "options": {
      "colorMode": "value",
      "graphMode": "area",
      "justifyMode": "auto",
      "orientation": "auto",
```

```
                "percentChangeColorMode": "standard",
                "reduceOptions": {
                    "calcs": [
                        "lastNotNull"
                    ],
                    "fields": "",
                    "values": false
                },
                "showPercentChange": false,
                "textMode": "auto",
                "wideLayout": true
            },
            "pluginVersion": "13.0.1",
            "targets": [
                {
                    "datasource": {
                        "type": "prometheus",
                        "uid": "prometheus"
                    },
                    "editorMode": "code",
                    "expr": "sum(kube_pod_status_phase{phase=~\"Pending|Failed|Unknown\", namespace=\"default\", pod=~\"backend.*|dashboard.*|postgres.*\"})",
                    "legendFormat": "__auto",
                    "range": true,
                    "refId": "A"
                }
            ],
            "title": "Problem pods",
            "type": "stat"
        }
    ],
    "preload": false,
    "refresh": "",
    "schemaVersion": 42,
    "time": {
        "from": "now-1h",
        "to": "now"
    },
    "timepicker": {
        "refresh_intervals": [
```

"5s",
"10s",
"30s",
"1m",
"5m",
"15m",
"30m",
"1h",
"2h",
"1d"
]
},
"timezone": "browser",
"title": "LabServer Kubernetes Monitoring",
"uid": "ad7mjnj",
"version": 6
}